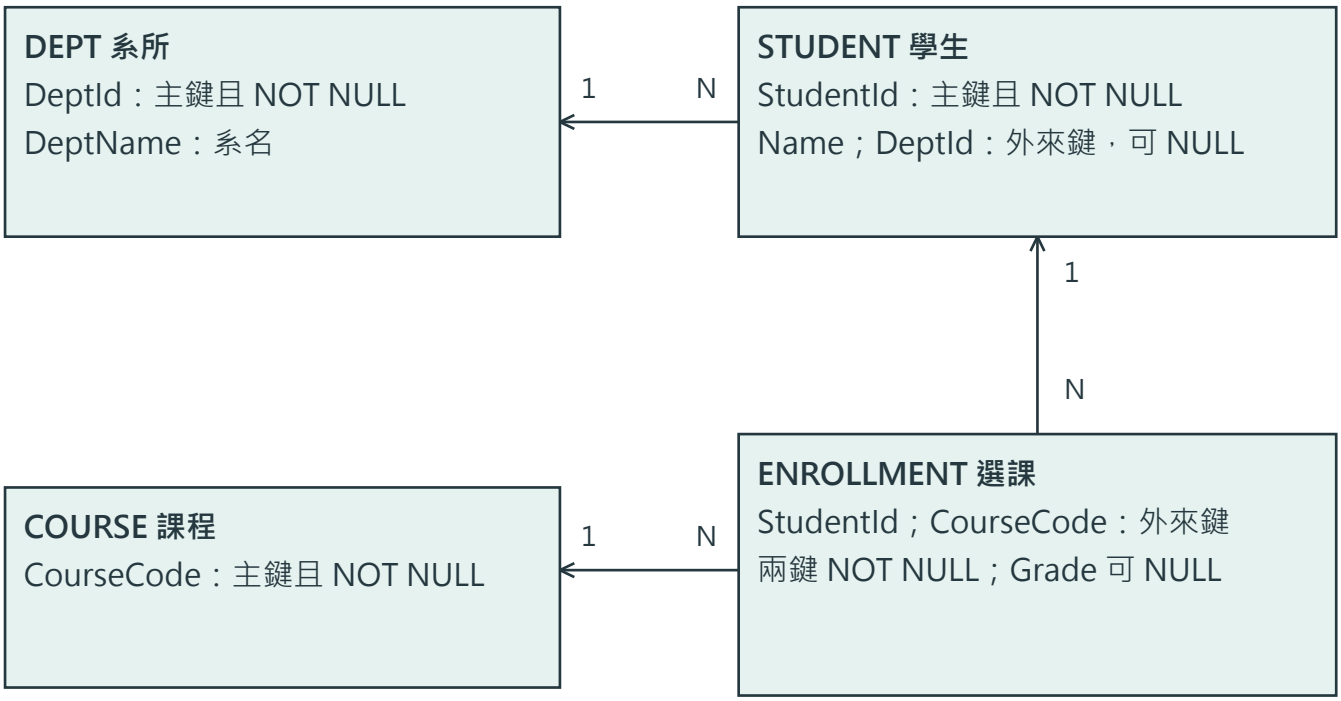
學校資料：系所、學生、課程與選課

教務處要查選課名單與成績。學生可以尚未分系、沒有選課；選課成績也可以尚未登錄。先看四張表如何相連。



箭頭從外來鍵指向主鍵。選課的 StudentId、CourseCode 合起來為主鍵；每筆選課連一位學生、一門課。

資料全為人工教學例。沿用 Ch6 的八位學生、八筆選課；本章只新增 D3 設計系，目前沒有學生。

問題：這章要回答什麼？

回答：哪些資料缺值、誰有或沒有配對、如何統計，再把查詢或異動規則保存起來。

本頁重點：先確認四表與連線，再開始操作；不能只用姓名識別學生。

完整起始資料：八位學生與三個系所

S 是學號，D 是系所代碼。S1、S2 都叫 Alex，是不同學生；學號不必連續，S4 不在本例。NULL 表示尚未分系，不是名叫 NULL 的系。

DeptId	DeptName
D1	資訊管理
D2	會計
D3	設計

D1 = 資訊管理；D2 = 會計；D3 = 設計。D3 尚無學生。

StudentId	Name	DeptId
S1	Alex	D1
S2	Alex	D1
S3	Casey	NULL
S5	Morgan	D1
S6	Riley	D2
S7	Taylor	D2
S8	Jordan	D2
S9	Robin	NULL

全部八人：S3、S9 尚未分系；D1 與 D2 各有三人。

本頁重點：代碼與姓名不同：同名不能合併，缺值不能當成不存在的學生。

完整起始資料：三門課與八筆選課

DB101 = 資料庫入門；CS102 = 程式設計；NW103 = 網路入門。Grade 是 0–100 的整數成績，NULL 表示尚未登錄。

COURSE.CourseCode
DB101
CS102
NW103

COURSE 全三列；課名只用中文解釋代碼，並非另加課名欄位。

StudentId	CourseCode	Grade
S1	CS102	NULL
S1	DB101	80
S2	DB101	0
S6	CS102	90
S6	NW103	95
S7	DB101	60
S8	DB101	75
S9	CS102	NULL

ENROLLMENT 全部八筆。S1、S6 各選兩門；S3、S5 沒有選課列。

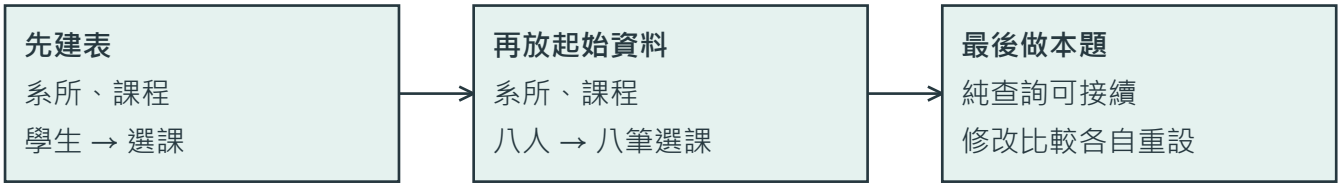
問題：S2 的 0 與 S9 的 NULL 一樣嗎？

回答：不一樣。0 是已知成績，NULL 是尚未登錄。

本頁重點：先找選課列，再看成績是否已登錄；缺成績不代表沒選課。

建置順序與資料重設

本章程式使用 SQLite。先在空資料庫依 第77頁到第82頁完成六步建置。之後 A、B 是同題的兩種寫法，從相同起始資料比較。



箭頭是準備順序，不是查詢的實際執行計畫。

每次連線先啟用 PRAGMA foreign_keys = ON。本教材統一用 NULL 顯示缺值；工具也可能顯示空白或 None。多列結果用 ORDER BY 固定展示順序。

本章「重設」是另開空資料庫，依六步重新建置；本題若要另外建立查詢或自動規則，再依該頁步驟準備。A、B 都只查詢時，可共用起始資料。

問題：哪一種比較必須分開重設？

回答：A 已修改資料或建好物件時，B 要重新準備；只做 SELECT 的比較可以接續執行。

本頁重點：先準備一次固定資料；修改型比較各自重設，純查詢比較可接續。

缺成績：IS NULL 與 = NULL

找出尚未登錄成績的選課。 從開場資料開始；A、B 都只查詢，可接續執行。

A：兩版各得幾列？S1、S9 的缺成績會被找到嗎？ 先預測，再執行。

```
SELECT StudentId, CourseCode, Grade
FROM ENROLLMENT WHERE Grade = NULL
ORDER BY StudentId, CourseCode;
```

B：同一問題，改用以下寫法。

```
SELECT StudentId, CourseCode, Grade
FROM ENROLLMENT WHERE Grade IS NULL
ORDER BY StudentId, CourseCode;
```

A：不符合本題目標

0 列
沒有結果列

B：修正後

學號	課程	成績
S1	CS102	NULL
S9	CS102	NULL

兩段查詢的實際完整結果。

答案：B 得兩列：S1、S9 的 CS102。A 得 0 列，因為 = NULL 的比較不是成立。

本頁重點：要找缺值，用 IS NULL；不是用等號和 NULL 比較。

成績條件：TRUE、FALSE 與 UNKNOWN

問題固定為 Grade >= 60 是否成立。未登錄成績時無法判定；SQL 稱為 UNKNOWN。WHERE 只保留成立的列。

學號 / 課程	Grade	>= 60	NOT (>= 60)
S1 / CS102	NULL	UNKNOWN	UNKNOWN
S1 / DB101	80	TRUE	FALSE
S2 / DB101	0	FALSE	TRUE
S6 / CS102	90	TRUE	FALSE
S6 / NW103	95	TRUE	FALSE
S7 / DB101	60	TRUE	FALSE
S8 / DB101	75	TRUE	FALSE
S9 / CS102	NULL	UNKNOWN	UNKNOWN

完整八列：五列成立、一列不成立、兩列無法判定。

```
SELECT StudentId, CourseCode, Grade
FROM ENROLLMENT WHERE NOT (Grade >= 60)
ORDER BY StudentId, CourseCode;
```

StudentId	CourseCode	Grade
S2	DB101	0

實際完整輸出只有這一列。

問題：NOT (Grade >= 60) 包含缺成績嗎？

回答：沒有。只有 S2 的 0 分；缺值取 NOT 仍是 UNKNOWN。

本頁重點：五列及格加一列未及格，不等於八列；另外兩列是尚未登錄。

非 D1 與未分系：用 OR 保留缺值

列出不是 D1 或尚未分系的學生。 從開場資料開始；A、B 都只查詢，可接續執行。

A：兩版會保留尚未分系的 S3、S9 嗎？各得幾人？ 先預測，再執行。

```
SELECT StudentId, DeptId FROM STUDENT
WHERE DeptId <> 'D1' ORDER BY StudentId;
```

B：同一問題，改用以下寫法。

```
SELECT StudentId, DeptId FROM STUDENT
WHERE DeptId <> 'D1' OR DeptId IS NULL
ORDER BY StudentId;
```

A：不符合本題目標

學號	系所
S6	D2
S7	D2
S8	D2

B：修正後

學號	系所
S3	NULL
S6	D2
S7	D2
S8	D2
S9	NULL

兩段查詢的實際完整結果。

答案：B 的五人符合這次明訂的目標。A 漏了尚未分系的 S3、S9。

本頁重點：「非 D1」是否包含未分系，要在問題中明說，再用 OR 加入缺值。

子查詢：先取得選課學號

要找有選課的學生，先從選課表取得學號，再看學生學號是否在其中。放在另一個查詢括號內的 SELECT 稱為子查詢。

```
SELECT StudentId FROM ENROLLMENT
ORDER BY StudentId, CourseCode;
```

子查詢學號
S1
S1
S2
S6
S6
S7
S8
S9

八筆選課提供八個學號值；S1、S6 重複，因為各選兩門。

IN 的意思是「在這些值中」。下一頁將這段 SELECT 放進 IN；s 代表學生表、e 代表選課表，AS 給表暫時的名稱。

本頁重點：子查詢先提供要比對的資料；不是新的表名，也不要求學號值都不同。

有選課的學生：JOIN 與 IN

每位有選課的學生只列一次。 從開場資料開始；A、B 都只查詢，可接續執行。

A：兩版的 S1、S6 各出現幾次？總共幾列？ 先預測，再執行。

```
SELECT s.StudentId, s.Name
FROM STUDENT AS s, ENROLLMENT AS e
WHERE s.StudentId = e.StudentId ORDER BY s.StudentId;
```

B：同一問題，改用以下寫法。

```
SELECT StudentId, Name FROM STUDENT
WHERE StudentId IN
  (SELECT StudentId FROM ENROLLMENT)
ORDER BY StudentId;
```

A：不符合本題目標

學號	姓名
S1	Alex
S1	Alex
S2	Alex
S6	Riley
S6	Riley
S7	Taylor
S8	Jordan
S9	Robin

B：修正後

學號	姓名
S1	Alex
S2	Alex
S6	Riley
S7	Taylor
S8	Jordan
S9	Robin

兩段查詢的實際完整結果。

答案：A 印八筆配對，S1、S6 各出現兩次。B 印六位學生，符合本題每人一次。

本頁重點：JOIN 產生配對列；IN 判斷這位學生是否在名單中。

DB101 子查詢：先看四個學號

教務處要列出所有選 DB101 的學生。先只查選課表；這段結果就是下一頁放進括號內的名單。

先預測：DB101 有幾筆選課？內層會回傳一個學號，還是多個學號？

```
SELECT StudentId FROM ENROLLMENT
WHERE CourseCode = 'DB101' ORDER BY StudentId;
```

StudentId
S1
S2
S7
S8

實際完整輸出四列；四個不同學號都必須保留。

問題：若只用第一個學號 S1 來篩學生，會漏掉誰？

回答：S2、S7、S8。題目要完整四人名單，不能只拿第一個值。

下一頁用 = 時，右邊當成一個值，稱單值子查詢；用 IN 時，右邊當成整份名單。先分清用途，再看 SQLite 的實作額外補充。

本頁重點：先看子查詢回傳幾列，再選擇符合題目需求的比較方式。

子查詢多個學號：= 與 IN

列出所有選 DB101 的學生。 從開場資料開始；A、B 都只查詢，可接續執行。= 右側當成一個值使用；IN 右側當成名單。實作額外補充（SQLite）：當成單值使用時，即使子查詢有多列，也只取第一列。

A：DB101 的四位學生，A、B 各能列出幾位？ 先預測，再執行。

```
SELECT StudentId FROM STUDENT
WHERE StudentId = (SELECT StudentId FROM ENROLLMENT
  WHERE CourseCode = 'DB101' ORDER BY StudentId)
ORDER BY StudentId;
```

B：同一問題，改用以下寫法。

```
SELECT StudentId FROM STUDENT
WHERE StudentId IN (SELECT StudentId FROM ENROLLMENT
  WHERE CourseCode = 'DB101' ORDER BY StudentId)
ORDER BY StudentId;
```

A：不符合本題目標

學號
S1

B：修正後

學號
S1
S2
S7
S8

兩段查詢的實際完整結果。

答案：A 只拿到 S1；B 列出 S1、S2、S7、S8。兩版只改 = / IN；內層排序固定 A 取 S1，排序不影響 IN 的名單判斷。

本頁重點：SQLite 實作補充：單值子查詢有多列時取第一列，不會在這裡報錯；這不能當作正確名單。

EXISTS：先逐人找相符選課

目標仍是找有選課的學生。s 代表目前檢查的學生，e 代表選課列；相同學號才算配對。有至少一筆就保留這位學生，這種檢查寫成 EXISTS。

外層學生 s	內層相符選課	EXISTS
S1	CS102、DB101	TRUE
S2	DB101	TRUE
S3	無	FALSE
S5	無	FALSE
S6	CS102、NW103	TRUE
S7	DB101	TRUE
S8	DB101	TRUE
S9	CS102	TRUE

逐一核對八位學生；S3、S5 沒有任何相符選課。這是判斷圖，不是實際執行計畫。

子查詢用 e.StudentId = s.StudentId 連到外層這位學生，稱為相關子查詢。下一頁 SELECT 1 的 1 只是常數；EXISTS 不在計數。

問題：S1 有兩筆，EXISTS 會印兩個 S1 嗎？

回答：不會；只判斷成立或不成立，外層學生仍是一列。

本頁重點：先認清別名與相連條件；不要把「全校有人選課」誤讀成「每人有選課」。

EXISTS：缺少學號條件會多列誰？

每位有選課的學生只列一次。 從開場資料開始；A、B 都只查詢，可接續執行。

A：沒有選課的 S3、S5，會出現在 A 或 B 嗎？ 先預測，再執行。

```
SELECT s.StudentId FROM STUDENT AS s
WHERE EXISTS (SELECT 1 FROM ENROLLMENT AS e)
ORDER BY s.StudentId;
```

B：同一問題，改用以下寫法。

```
SELECT s.StudentId FROM STUDENT AS s
WHERE EXISTS (SELECT 1 FROM ENROLLMENT AS e
  WHERE e.StudentId = s.StudentId) ORDER BY s.StudentId;
```

A：不符合本題目標

學號
S1
S2
S3
S5
S6
S7
S8
S9

B：修正後

學號
S1
S2
S6
S7
S8
S9

兩段查詢的實際完整結果。

答案：A 對每人都看同一份全校八筆選課，所以八人全列出。B 加入學號相等條件，只保留六位真的有選課的學生。

本頁重點：兩版只差相連條件；S3、S5 是判斷有沒有回答原題的反例。

沒有選課：NOT EXISTS 與 NOT IN

列出沒有任何選課的學生。NOT IN 檢查學號不在選課名單中；NOT EXISTS 檢查找不到相同學號的選課列。A、B 都只查詢同一起始資料。

A：哪兩位學生沒有選課？兩版名單會相同嗎？ 先預測，再執行。

```
SELECT StudentId FROM STUDENT
WHERE StudentId NOT IN
  (SELECT StudentId FROM ENROLLMENT) ORDER BY StudentId;
```

B：同一問題，改用以下寫法。

```
SELECT s.StudentId FROM STUDENT AS s
WHERE NOT EXISTS (SELECT 1 FROM ENROLLMENT AS e
  WHERE e.StudentId = s.StudentId) ORDER BY s.StudentId;
```

A：NOT IN，正確

學號
S3
S5

B：NOT EXISTS，正確

學號
S3
S5

兩段查詢的實際完整結果。

答案：兩版這次都得到 S3、S5。選課表 StudentId 已設 NOT NULL，名單中不會有 NULL。

本頁重點：這頁 A 也正確；下一頁改用可缺值的 DeptId，才會看見 NOT IN 的問題。

沒有學生的系所：先看兩份資料

教務處要找尚無學生的系所。D3 沒有學生；S3、S9 尚未分系，不能算在 D3。以下都只查詢，可依頁序接著執行，不修改起始資料。

DEPT.DeptId	DeptName
D1	資訊管理
D2	會計
D3	設計

外層要檢查三個系所；DeptId 是 NOT NULL 主鍵。

StudentId	STUDENT.DeptId
S1	D1
S2	D1
S3	NULL
S5	D1
S6	D2
S7	D2
S8	D2
S9	NULL

完整八位學生。下一段子查詢只取右欄，保留重複值與兩個 NULL。

```
SELECT DeptId FROM STUDENT ORDER BY StudentId;
```

實際輸出由上到下：D1、D1、NULL、D1、D2、D2、D2、NULL，共八列。

本頁重點：問題的答案應為 D3。先用這八個值測試 IN，再逐步改成 NOT IN。

加入 IN：先顯示判斷，不篩選系所

把前頁 SELECT 放進 IN 的括號。這一步將判斷放在 SELECT 輸出欄中，還沒用 WHERE 篩掉系所。先猜 D3 的判斷是 0，還是 NULL。

```
SELECT DeptId,
  DeptId IN (SELECT DeptId FROM STUDENT) AS InList
FROM DEPT ORDER BY DeptId;
```

DeptId	InList
D1	1
D2	1
D3	NULL

完整輸出仍是三列。SQLite 用 1 表示 TRUE、0 表示 FALSE、NULL 表示 UNKNOWN。

系所	對照八個值	IN 的判斷
D1	有相同的 D1	1
D2	有相同的 D2	1
D3	無相同已知值，但含 NULL	NULL

這是結果說明表，不是另一段 SQL 輸出。

問題：為何 D3 不是 0？

回答：D3 與已知 D1、D2 都不同，但與 NULL 比較無法判定；整個 IN 因而是 UNKNOWN。

本頁重點：SELECT 顯示判斷，不會因為判斷為 NULL 就移除這一列。

加入 NOT：1 變 0，NULL 仍是 NULL

同樣三個系所、同樣八個內層值。只把 IN 改成 NOT IN；AS NoStudent 是輸出欄名，不是預先保證查詢正確。先猜 D3 會不會變成 1。

```
SELECT DeptId,
  DeptId NOT IN (SELECT DeptId FROM STUDENT) AS NoStudent
FROM DEPT ORDER BY DeptId;
```

DeptId	NoStudent
D1	0
D2	0
D3	NULL

實際完整輸出三列，尚未篩選。

系所	前頁 IN	本頁 NOT IN
D1	1	0
D2	1	0
D3	NULL	NULL

並排對照兩次輸出；NOT 不會把 UNKNOWN 變成 TRUE。

問題：D3 明明沒有人，這段判斷有得到 1 嗎？

回答：沒有，仍是 NULL。下一頁才把這個判斷放進 WHERE，查看留下哪些列。

本頁重點：目前問題不在資料消失，而在 D3 的 NOT IN 判斷沒有成立。

加入 WHERE：三個系所都沒有留下

把前頁相同的 NOT IN 放進 WHERE。WHERE 只保留判斷為 1 的列；0 與 NULL 都不保留。先從前頁三個判斷值預測輸出列數。

```
SELECT DeptId FROM DEPT
WHERE DeptId NOT IN
  (SELECT DeptId FROM STUDENT) ORDER BY DeptId;
```

DeptId	NOT IN 判斷	WHERE 是否保留
D1	0	否
D2	0	否
D3	NULL	否

三個候選列如何被篩選；這是逐列說明，不是查詢輸出。

實際結果：0 列
沒有任何 DeptId 留下；不是一列 NULL。

D3 應在答案中，卻被漏掉。因此這段 SQL 不符合本題需求。

問題：DEPT 或 STUDENT 的原資料被刪掉了嗎？

回答：沒有。SELECT 只查詢；三個系所、八位學生都還在。

本頁重點：不是執行錯誤：SQL 可執行，但含 NULL 的名單使本題得到錯誤的空結果。

修正內層：只取已知的系所代碼

目標是找沒有已分系學生的系所。先單獨執行內層 SELECT，只加入 WHERE DeptId IS NOT NULL。預測八個值會剩下幾列。

```
SELECT DeptId FROM STUDENT
WHERE DeptId IS NOT NULL ORDER BY StudentId;
```

DeptId
D1
D1
D1
D2
D2
D2

實際完整輸出六列：D1 三列、D2 三列。這一步沒有加 DISTINCT，重複值仍保留。

未進入子查詢結果的學生	DeptId
S3	NULL
S9	NULL

八列減去這兩列，得到六列；學生原資料仍保留。

問題：這是不是把未分系的人改為 D1？

回答：不是。只在本次查詢排除缺值，沒有 UPDATE，也沒有 DELETE。

本頁重點：先確認內層六個值，再放回 NOT IN；不要同時改資料與查詢條件。

放回 NOT IN：D3 的判斷變成 1

只把內層換成前頁的六個已知系所值。外層仍先顯示判斷，不篩選；預測 D1、D2、D3 哪一個判斷會改變。

```
SELECT DeptId, DeptId NOT IN
  (SELECT DeptId FROM STUDENT WHERE DeptId IS NOT NULL)
  AS NoStudent
FROM DEPT ORDER BY DeptId;
```

DeptId	NoStudent
D1	0
D2	0
D3	1

實際完整輸出三列。D3 與六個已知值全部不同，NOT IN 現在成立。

系所	內層含 NULL	內層已排除 NULL
D1	0	0
D2	0	0
D3	NULL	1

只改內層 WHERE；D1、D2 不變，D3 從無法判定變為成立。

問題：下一步把這個條件放進外層 WHERE，會留下誰？

回答：只留 D3，因為只有它的判斷是 1。

本頁重點：先看中間判斷表，確認修正改變的是 D3，再組成最後查詢。

完整修正版：WHERE 留下 D3

同一問題的答案應只有 D3。內層 WHERE 排除未知系所；外層 WHERE 保留沒有相同已知學生系所的系。這兩個 WHERE 處理不同的列。

```
SELECT DeptId FROM DEPT
WHERE DeptId NOT IN (SELECT DeptId FROM STUDENT
  WHERE DeptId IS NOT NULL) ORDER BY DeptId;
```

外層 DeptId	NOT IN 判斷	保留？
D1	0	否
D2	0	否
D3	1	是

前頁的三個判斷值，現在用於外層篩選。

DeptId
D3

實際完整輸出一列，符合本題目標。

條件放在哪裡	處理哪份資料	結果
內層 STUDENT	八個學生系所值	六個非 NULL 值
外層 DEPT	三個系所	D3 一列

原資料不變；排除的是本次查詢中的候選列。

本頁重點：此題外層主鍵不可缺值，內層也已排除 NULL；NOT IN 因而得到正確答案。

同一問題再比較：原寫法與 NOT EXISTS

仍找沒有學生的系所；回看原本有問題的寫法。 從開場資料開始；A、B 都只查詢，可接續執行。

A：D3 沒有學生；A、B 都能列出 D3 嗎？ 先預測，再執行。

```
SELECT DeptId FROM DEPT
WHERE DeptId NOT IN
  (SELECT DeptId FROM STUDENT) ORDER BY DeptId;
```

B：同一問題，改用以下寫法。

```
SELECT d.DeptId FROM DEPT AS d
WHERE NOT EXISTS (SELECT 1 FROM STUDENT AS s
  WHERE s.DeptId = d.DeptId) ORDER BY d.DeptId;
```

A：不符合本題目標

0 列
沒有結果列

B：修正後

系所
D3

兩段查詢的實際完整結果。

答案：A 重現 0 列。B 逐系找相同 DeptId 的學生：D1、D2 找得到，D3 找不到，所以只留 D3。

本頁重點：第21頁已排除 NULL 的 NOT IN 也正確；本頁 B 是另一種正確寫法，不表示所有 NOT IN 都錯。

自我接合：固定 Riley，再列八位比較對象

找 Riley（S6）的其他同系同學。a、b 都來自 STUDENT：a 是要查的人，b 是比較對象。先只固定 a = S6；這一步還不判斷同系。

```
SELECT a.StudentId AS SelfId, b.StudentId AS PeerId, b.DeptId
FROM STUDENT AS a, STUDENT AS b
WHERE a.StudentId = 'S6' ORDER BY b.StudentId;
```

SelfId（a）	PeerId（b）	b.DeptId
S6	S1	D1
S6	S2	D1
S6	S3	NULL
S6	S5	D1
S6	S6	D2
S6	S7	D2
S6	S8	D2
S6	S9	NULL

實際完整輸出八列；Riley 在 D2。同一表的兩個角色也可以配到同一人，例如 (S6,S6)。

尚未篩選時有 8 × 8 = 64 組；限定 a = S6 後為 1 × 8 = 8 組。這是配對數的推導，不是資料庫的實際執行計畫。

問題：八組中哪些 b 在 D2？

回答：S6、S7、S8，共三組；其中 S6 就是 Riley 本人。

本頁重點：FROM 的兩個別名不會自動排除自己；下一頁再比較是否排除本人。

自我接合：同系條件還要排除本人

列出 Riley（S6）的其他同系同學；Riley 自己不能出現在同學名單。 從開場資料開始；A、B 都只查詢，可接續執行。

A：同系有 S6、S7、S8；A、B 會把 Riley 本人也列出嗎？ 先預測，再執行。

```
SELECT a.StudentId AS SelfId, b.StudentId AS PeerId
FROM STUDENT AS a, STUDENT AS b
WHERE a.StudentId = 'S6' AND a.DeptId = b.DeptId
ORDER BY b.StudentId;
```

B：同一問題，改用以下寫法。

```
SELECT a.StudentId AS SelfId, b.StudentId AS PeerId
FROM STUDENT AS a, STUDENT AS b
WHERE a.StudentId = 'S6' AND a.DeptId = b.DeptId
  AND a.StudentId <> b.StudentId ORDER BY b.StudentId;
```

A：不符合本題目標

SelfId	PeerId
S6	S6
S6	S7
S6	S8

B：修正後

SelfId	PeerId
S6	S7
S6	S8

兩段查詢的實際完整結果。

答案：A 的三組包含 (S6,S6)；B 只多學號不同條件，剩下 (S6,S7)、(S6,S8)。S3、S9 未分系，也不會被等號配成 D2。

本頁重點：固定本人留下八組，同系留下三組，排除本人剩兩組；每個條件各做一件事。

全部學生的選課：INNER JOIN 會漏誰？

目標是全部八位學生都要出現。INNER JOIN 只列出配對成功的列，ON 指定學號相等。先預測沒有選課的 S3、S5 能否出現。

```
SELECT s.StudentId, e.CourseCode, e.Grade
FROM STUDENT AS s INNER JOIN ENROLLMENT AS e
ON s.StudentId = e.StudentId
ORDER BY s.StudentId, e.CourseCode;
```

本例與先前 FROM STUDENT AS s, ENROLLMENT AS e 加 WHERE 學號相等的配對相同，現在把配對條件寫在 ON。單寫 JOIN 也是 INNER JOIN；不要把這個改寫直接套到 LEFT JOIN。

學號	課程	成績
S1	CS102	NULL
S1	DB101	80
S2	DB101	0
S6	CS102	90
S6	NW103	95
S7	DB101	60
S8	DB101	75
S9	CS102	NULL

完整八筆配對。八列不等於八位學生：S1、S6 重複，S3、S5 完全沒有出現。

問題：怎麼保留沒有選課的兩人？

回答：下一頁把 JOIN 改成 LEFT JOIN。

本頁重點：INNER JOIN 只保留有配對的列，不符合本題「全部學生」的目標。

全部學生的選課：LEFT JOIN 保留未選課者

左側 STUDENT 保留全部八人；沒有右側選課配對時，右側欄位補 NULL。只改前頁的接合方式。

```
SELECT s.StudentId, e.CourseCode, e.Grade
FROM STUDENT AS s LEFT JOIN ENROLLMENT AS e
ON s.StudentId = e.StudentId
ORDER BY s.StudentId, e.CourseCode;
```

學號	課程	成績
S1	CS102	NULL
S1	DB101	80
S2	DB101	0
S3	NULL	NULL
S5	NULL	NULL
S6	CS102	90
S6	NW103	95
S7	DB101	60
S8	DB101	75
S9	CS102	NULL

完整十列：原本八筆配對，加 S3、S5 各一列。S9 有 CS102，不是補出的未選課列。

問題：如何分辨沒有選課和只是缺成績？

回答：S3、S5 的 CourseCode 也是 NULL；S9 的 CourseCode 是 CS102。

本頁重點：LEFT JOIN 保留左表的每位學生，不代表結果一定只有八列。

WHERE 前：十列接合結果如何判斷？

這題改為保留全部八位學生，只顯示 DB101。先沿用上一頁十列，增加 IsDB101 判斷欄；還沒有 WHERE，不會篩掉任何列。

先預測：S3 沒有選課，S6 只選別課；IsDB101 會分別顯示 NULL、0，還是 1？

```
SELECT s.StudentId, e.CourseCode, e.Grade,
  e.CourseCode = 'DB101' AS IsDB101
FROM STUDENT AS s LEFT JOIN ENROLLMENT AS e
ON s.StudentId = e.StudentId
ORDER BY s.StudentId, e.CourseCode;
```

StudentId	CourseCode	Grade	IsDB101
S1	CS102	NULL	0
S1	DB101	80	1
S2	DB101	0	1
S3	NULL	NULL	NULL
S5	NULL	NULL	NULL
S6	CS102	90	0
S6	NW103	95	0
S7	DB101	60	1
S8	DB101	75	1
S9	CS102	NULL	0

SQLite 實作顯示：1 為成立、0 為不成立、NULL 為未知。完整十列都還在。

問題：若再加 WHERE e.CourseCode = 'DB101'，留下誰？

回答：只留下判斷為 1 的 S1、S2、S7、S8，共四列。S3、S5 的 NULL 也不會通過。

本頁重點：SELECT 顯示判斷值；WHERE 才只留下條件成立的列。

WHERE 加課程條件：十列只剩四列

目標仍是八位學生都要出現，只顯示 DB101。這次把上一頁的課程判斷放進 WHERE；接合條件仍只比較學號。

先預測：IsDB101 為 0 或 NULL 的列會留下嗎？

```
SELECT s.StudentId, e.CourseCode, e.Grade
FROM STUDENT AS s LEFT JOIN ENROLLMENT AS e
ON s.StudentId = e.StudentId
WHERE e.CourseCode = 'DB101' ORDER BY s.StudentId;
```

StudentId	CourseCode	Grade
S1	DB101	80
S2	DB101	0
S7	DB101	60
S8	DB101	75

實際完整結果四列，只保留原判斷為 1 的列。

被篩掉的列	CourseCode	原判斷
S1	CS102	0
S3	NULL	NULL
S5	NULL	NULL
S6	CS102	0
S6	NW103	0
S9	CS102	0

十列減去這六列，剩四列；原本八位學生和八筆選課都沒有被刪除。

問題：這段查詢是否符合八位學生都要出現？

回答：不符合。S3、S5、S6、S9 完全消失；下一頁把課程條件移到 ON，保留找不到 DB101 的學生。

本頁重點：WHERE 篩的是接合後的列，連未配對而補 NULL 的列也會被篩掉。

ON 加課程條件：仍保留全部八位學生

仍使用原本八位學生與八筆選課。移除前頁 WHERE，把 DB101 條件放入 ON；每人只找這門課，找不到仍保留學生。

先預測：S6 原有 CS102、NW103 兩筆；本次會出現兩列、一列，還是零列？

```
SELECT s.StudentId, e.CourseCode, e.Grade
FROM STUDENT AS s LEFT JOIN ENROLLMENT AS e
ON s.StudentId = e.StudentId AND e.CourseCode = 'DB101'
ORDER BY s.StudentId;
```

StudentId	CourseCode	Grade
S1	DB101	80
S2	DB101	0
S3	NULL	NULL
S5	NULL	NULL
S6	NULL	NULL
S7	DB101	60
S8	DB101	75
S9	NULL	NULL

完整八列：四筆 DB101 配對，加上 S3、S5、S6、S9 各一筆補出的列。

問題：S6 的 CourseCode 為 NULL，能說他完全沒選課嗎？

回答：不能。他只是沒有 DB101 配對；原本的 CS102、NW103 並未被刪除。

本頁重點：ON 限定右側配對；LEFT JOIN 保留沒有符合配對的左側學生。

每位學生的 DB101 成績：條件放 ON 或 WHERE

全部學生都要出現，只配對 DB101。A、B 均查原始資料，可接續執行；比較條件位置。

A：S6 只選別的課；A、B 會保留 S6 嗎？ 先預測，再執行。

```
SELECT s.StudentId, e.CourseCode, e.Grade
FROM STUDENT AS s LEFT JOIN ENROLLMENT AS e
ON s.StudentId = e.StudentId
WHERE e.CourseCode = 'DB101' ORDER BY s.StudentId;
```

B：同一問題，改用以下寫法。

```
SELECT s.StudentId, e.CourseCode, e.Grade
FROM STUDENT AS s LEFT JOIN ENROLLMENT AS e
ON s.StudentId = e.StudentId AND e.CourseCode = 'DB101'
ORDER BY s.StudentId;
```

A：不符合本題目標

學號	課程	成績
S1	DB101	80
S2	DB101	0
S7	DB101	60
S8	DB101	75

B：修正後

學號	課程	成績
S1	DB101	80
S2	DB101	0
S3	NULL	NULL
S5	NULL	NULL
S6	NULL	NULL
S7	DB101	60
S8	DB101	75
S9	NULL	NULL

兩段查詢的實際完整結果。

答案：A 留 DB101 四列，排除沒課程的 S3、S5，以及只選別課的 S6、S9；B 保留八人。

本頁重點：這頁的 NULL 是沒有 DB101 配對，不代表完全沒選課。ON 決定配對；WHERE 再篩接合結果。

選課數與已登錄成績數：COUNT 的差異

先預測八筆選課中有幾筆已登錄成績。把多列整理成一個數字，稱為聚合；COUNT 是計數函數。

```
SELECT COUNT(*) AS RowCnt, COUNT(Grade) AS Known
FROM ENROLLMENT;
```

逐列拆解：下表說明每筆如何計數，不是 SQL 輸出。

學號 / 課程	Grade	計列數	計已知成績
S1 / CS102	NULL	1	0
S1 / DB101	80	1	1
S2 / DB101	0	1	1
S6 / CS102	90	1	1
S6 / NW103	95	1	1
S7 / DB101	60	1	1
S8 / DB101	75	1	1
S9 / CS102	NULL	1	0

八個 1 相加是列數 8；最後一欄相加是已知成績 6，0 分也要計入。

RowCnt	Known
8	6

完整結果一列。

本頁重點：COUNT(*) 計全部列；COUNT(Grade) 只計 Grade 非 NULL 的列。

CS102 平均：除以三筆，還是已知的一筆？

計算 CS102 已登錄成績的平均。 從開場資料開始；A、B 都只查詢，可接續執行。CS102 是 NULL、90、NULL。SUM 加總已知值；AVG 計算已知值平均。

A：兩版的平均各是多少？分母包含幾筆成績？ 先預測，再執行。

```
SELECT SUM(Grade) * 1.0 / COUNT(*) AS Mean
FROM ENROLLMENT WHERE CourseCode = 'CS102';
```

B：同一問題，改用以下寫法。

```
SELECT AVG(Grade) AS Mean
FROM ENROLLMENT WHERE CourseCode = 'CS102';
```

A：不符合本題目標

平均
30.0

B：修正後

平均
90.0

兩段查詢的實際完整結果。

答案：CS102 成績是 NULL、90、NULL。SUM 為 90；A 除以三筆得 30，B 除以一筆已知值得 90。

本頁重點：NULL 不當成 0；* 1.0 使 A 使用小數除法。

沒有已知成績：平均是 NULL，不是 0

固定同一份選課資料。A 查 S9；B 查 DB101 中小於 0 的成績。先看 WHERE 留下哪些輸入，再預測四個聚合值。

A：WHERE 留下1列

學號	課程	Grade
S9	CS102	NULL

B：WHERE 留下0列

沒有符合的列
DB101：80、0、60、75
沒有任何成績小於0

上圖是聚合前的輸入，不是下面兩段 SQL 的輸出。

A：計算 S9 這一列。

```
SELECT COUNT(*), COUNT(Grade), SUM(Grade), AVG(Grade)
FROM ENROLLMENT WHERE StudentId = 'S9';
```

B：相同聚合，只改 WHERE。

```
SELECT COUNT(*), COUNT(Grade), SUM(Grade), AVG(Grade)
FROM ENROLLMENT WHERE CourseCode = 'DB101' AND Grade < 0;
```

版本	COUNT(*)	COUNT(Grade)	SUM(Grade)	AVG(Grade)
A	1	0	NULL	NULL
B	0	0	NULL	NULL

A、B 各回傳一列，合併在此對照；版本是對照標籤，不是 SQL 輸出欄。

問題：B 輸入0列，為何仍有一列答案？

回答：本頁沒有 GROUP BY，聚合仍回傳一列。COUNT 的0是筆數，不是0分；SUM、AVG 沒有已知值可算，所以是 NULL。

本頁重點：A 有一列但成績缺值；B 連輸入列都沒有。兩者 COUNT(*) 不同，AVG 都不能寫成0分。

每門課的平均：GROUP BY

先把同一 CourseCode 的選課放在同一組，再分組計算。GROUP BY 指定用哪個欄位分組。

```
SELECT CourseCode, COUNT(*), COUNT(Grade), AVG(Grade)
FROM ENROLLMENT GROUP BY CourseCode ORDER BY CourseCode;
```

CS102
NULL、90、NULL
90 / 1 = 90

DB101
80、0、60、75
215 / 4 = 53.75

NW103
95
95 / 1 = 95

三個獨立分組，各算自己的平均；DB101 的 0 不能略掉。

課程	選課數	已知數	平均
CS102	3	1	90.0
DB101	4	4	53.75
NW103	1	1	95.0

完整三組結果。

本頁重點：GROUP BY 不會自動去掉缺值列；COUNT(Grade)、AVG(Grade) 會略過 Grade 的 NULL。

分組識別學生：姓名或學號

分別計算兩位 Alex 的選課數。 從開場資料開始；A、B 都只查詢，可接續執行。

A：兩位 Alex 會被分成幾組？每組會計到幾筆？ 先預測，再執行。

```
SELECT s.Name, COUNT(*) FROM STUDENT AS s
JOIN ENROLLMENT AS e ON s.StudentId = e.StudentId
WHERE s.Name = 'Alex' GROUP BY s.Name ORDER BY s.Name;
```

B：同一問題，改用以下寫法。

```
SELECT s.StudentId, COUNT(*) FROM STUDENT AS s
JOIN ENROLLMENT AS e ON s.StudentId = e.StudentId
WHERE s.Name = 'Alex' GROUP BY s.StudentId
ORDER BY s.StudentId;
```

A：不符合本題目標

分組值	選課數
Alex	3

B：修正後

分組值	選課數
S1	2
S2	1

兩段查詢的實際完整結果。

答案：A 合成一組 Alex 三筆；B 保留 S1 的兩筆與 S2 的一筆。

本頁重點：分組欄位會決定誰被合在一起；姓名不是這份資料的唯一識別。

每位學生的選課數：不要數補出的列

列出每位學生實際的選課數，未選課者為 0。 從開場資料開始；A、B 都只查詢，可接續執行。

A：S3、S5 沒有選課；兩版會各算成 0 還是 1？ 先預測，再執行。

```
SELECT s.StudentId, COUNT(*) FROM STUDENT AS s
LEFT JOIN ENROLLMENT AS e ON s.StudentId = e.StudentId
GROUP BY s.StudentId ORDER BY s.StudentId;
```

B：同一問題，改用以下寫法。

```
SELECT s.StudentId, COUNT(e.CourseCode) FROM STUDENT AS s
LEFT JOIN ENROLLMENT AS e ON s.StudentId = e.StudentId
GROUP BY s.StudentId ORDER BY s.StudentId;
```

A：不符合本題目標

學號	選課數
S1	2
S2	1
S3	1
S5	1
S6	2
S7	1
S8	1
S9	1

B：修正後

學號	選課數
S1	2
S2	1
S3	0
S5	0
S6	2
S7	1
S8	1
S9	1

兩段查詢的實際完整結果。

答案：A 把 S3、S5 的補出列各算一次。B 計 CourseCode，兩人為 0；S9 有 CS102，只是成績 NULL，所以仍是 1。

本頁重點：要數實際配對，計右側已設非 NULL 的鍵，不要計 LEFT JOIN 的補出列。

系所分組：未分系的兩人會同組

GROUP BY DeptId 將已知的系分組；缺值在分組時放同一組。這不等於已確認 S3 和 S9 同系。

```
SELECT DeptId, COUNT(*) FROM STUDENT
GROUP BY DeptId ORDER BY DeptId;
```

分組值	學生（說明）	人數
NULL	S3、S9	2
D1	S1、S2、S5	3
D2	S6、S7、S8	3

學生欄是圖解，不是 SELECT 輸出；完整輸出為 NULL / 2、D1 / 3、D2 / 3。D3 沒有學生列。

問題：前面用 a.DeptId = b.DeptId 能把 S3、S9 配成同系嗎？

回答：不能。分組的缺值處理，和等號比較的 UNKNOWN 不同。

本頁重點：GROUP BY 的 NULL 組是整理未知資料，不是給他們新設一個系所。

WHERE 先篩成績：平均之前少了哪四筆？

目標是找「整門課已登錄平均至少 70」的課程。先試一種可能有問題的做法：從原本八筆選課，只留下個別成績至少 70 的列，尚未分組。

先預測：DB101 的 0、60 會留下嗎？之後用剩下的成績，還能算出整門課平均嗎？

```
SELECT StudentId, CourseCode, Grade
FROM ENROLLMENT WHERE Grade >= 70
ORDER BY StudentId, CourseCode;
```

StudentId	CourseCode	Grade
S1	DB101	80
S6	CS102	90
S6	NW103	95
S8	DB101	75

完整結果四列。這是查詢篩選，沒有刪除原選課表。

沒有保留的選課	成績	未通過的原因
S2 / DB101	0	0 < 70
S7 / DB101	60	60 < 70
S1 / CS102	NULL	比較為 UNKNOWN
S9 / CS102	NULL	比較為 UNKNOWN

原八筆 = 上方四筆 + 這四筆。低分被排除後，DB101 只剩 80、75。

問題：把剩下的 DB101 算平均，結果是多少？

回答：(80 + 75) / 2 = 77.5。這只代表留下的兩筆，不是整門課四筆的平均。

本頁重點：本題不能先排除低分，再宣稱算的是整門課平均。

先算各課平均：三組如何判斷至少 70？

改從原本完整八筆選課開始，不使用前頁篩出的四筆。先 GROUP BY CourseCode 計算每課平均，再增加 KeepGroup 判斷欄；本頁還沒篩組。

先預測：DB101 的 80、0、60、75 全部計入後，能通過平均至少 70 嗎？

```
SELECT CourseCode, AVG(Grade) AS Mean,
  AVG(Grade) >= 70 AS KeepGroup
FROM ENROLLMENT GROUP BY CourseCode
ORDER BY CourseCode;
```

CourseCode	Mean	KeepGroup
CS102	90.0	1
DB101	53.75	0
NW103	95.0	1

完整三組都顯示；KeepGroup 的 1 表示成立，0 表示不成立。

課程	已知成績的計算	平均
CS102	90 / 1	90.0
DB101	(80 + 0 + 60 + 75) / 4	53.75
NW103	95 / 1	95.0

CS102 的兩筆 NULL 不計入平均分母；DB101 的 0 是已知成績，必須計入。

問題：下一步只保留 KeepGroup 為 1 的組，會剩哪些課？

回答：CS102、NW103。SQL 用 HAVING AVG(Grade) >= 70 篩分組，下一頁列出完整寫法。

本頁重點：先保留各課原有成績算平均，再用 HAVING 決定哪些組留下。

課程平均至少 70：WHERE 與 HAVING

列出整門課已登錄平均至少 70 的課程。前兩頁已看見 A 先少了低分、B 先保留完整成績；現在比較兩段完整 SQL。A、B 都從原八筆選課查詢。

A：DB101 在兩版的平均是多少？會被留下嗎？ 先預測，再執行。

```
SELECT CourseCode, AVG(Grade) FROM ENROLLMENT
WHERE Grade >= 70 GROUP BY CourseCode ORDER BY CourseCode;
```

B：同一問題，改用以下寫法。

```
SELECT CourseCode, AVG(Grade) FROM ENROLLMENT
GROUP BY CourseCode HAVING AVG(Grade) >= 70
ORDER BY CourseCode;
```

A：不符合本題目標

課程	平均
CS102	90.0
DB101	77.5
NW103	95.0

B：修正後

課程	平均
CS102	90.0
NW103	95.0

兩段查詢的實際完整結果。

答案：A 先排除 DB101 的 0、60，只算 (80+75)/2 = 77.5。B 先算完整平均 53.75，排除 DB101。

本頁重點：WHERE 篩個別列；HAVING 篩算完的組。兩者不是可互換的位置。

同課平均：先追 S7 的一筆選課

要找高於自己那門課平均的選課。先只檢查 S7：他的選課是 DB101、60 分；把這筆稱為外層 a。

a.StudentId	a.CourseCode	a.Grade
S7	DB101	60

目前只判斷這一筆；內層 b 仍讀原本完整選課表。

先預測：b 要取全校八筆，還是只取 a 所在的 DB101？

```
SELECT b.StudentId, b.CourseCode, b.Grade
FROM ENROLLMENT AS b
WHERE b.CourseCode = 'DB101' ORDER BY b.StudentId;
```

b.StudentId	b.CourseCode	b.Grade
S1	DB101	80
S2	DB101	0
S7	DB101	60
S8	DB101	75

完整四列；包含 S7 本人，因為題目要整門課平均，不是其他同學的平均。

問題：S7 要和多少分比較？會通過嗎？

回答：(80 + 0 + 60 + 75) / 4 = 53.75；60 > 53.75，所以保留 S7 / DB101。

這一筆的 a.CourseCode 是 DB101；下一頁用 b.CourseCode = a.CourseCode，讓每筆選課各自帶入課程。

本頁重點：先追一筆的課程、內層資料與平均，再推廣到全部八筆。

同課平均：先替八筆選課列出比較值

目標是找高於自己那門課平均的選課。a、b 都讀同一張 ENROLLMENT：a 是目前這筆選課，b 只取與它相同課程的列來算平均。

先預測：S7 / DB101 的 60 分，應該比較全校平均，還是 DB101 的 53.75？

```
SELECT a.StudentId, a.CourseCode, a.Grade,
  (SELECT AVG(b.Grade) FROM ENROLLMENT AS b
   WHERE b.CourseCode = a.CourseCode) AS CourseMean
FROM ENROLLMENT AS a ORDER BY a.StudentId, a.CourseCode;
```

StudentId	CourseCode	Grade	CourseMean
S1	CS102	NULL	90.0
S1	DB101	80	53.75
S2	DB101	0	53.75
S6	CS102	90	90.0
S6	NW103	95	95.0
S7	DB101	60	53.75
S8	DB101	75	53.75
S9	CS102	NULL	90.0

本步只顯示比較值，尚未篩選；上表完整八列，同課的列使用相同平均。

問題：逐列比較 Grade > CourseMean，哪些選課成立？

回答：S1 / DB101：80 > 53.75；S7 / DB101：60 > 53.75；S8 / DB101：75 > 53.75。90、95 都等於自己課程平均；NULL 無法判斷。

本頁重點：先確認每列拿到哪一門課的平均，再把同一個子查詢放進 WHERE 比較。

高於平均：全校平均或該課平均

找出高於自己那門課平均的選課。前頁已列出每筆的同課平均；現在比較 A 的全校平均與 B 的同課平均。a、b 仍代表同一張 ENROLLMENT 的兩個用途。

A：S7 的 DB101 為 60；兩版會留下這筆嗎？ 先預測，再執行。

```
SELECT StudentId, CourseCode, Grade FROM ENROLLMENT
WHERE Grade > (SELECT AVG(Grade) FROM ENROLLMENT)
ORDER BY StudentId, CourseCode;
```

B：同一問題，改用以下寫法。

```
SELECT a.StudentId, a.CourseCode, a.Grade
FROM ENROLLMENT AS a
WHERE a.Grade > (SELECT AVG(b.Grade) FROM ENROLLMENT AS b
  WHERE b.CourseCode = a.CourseCode)
ORDER BY a.StudentId, a.CourseCode;
```

A：不符合本題目標

學號	課程	成績
S1	DB101	80
S6	CS102	90
S6	NW103	95
S8	DB101	75

B：修正後

學號	課程	成績
S1	DB101	80
S7	DB101	60
S8	DB101	75

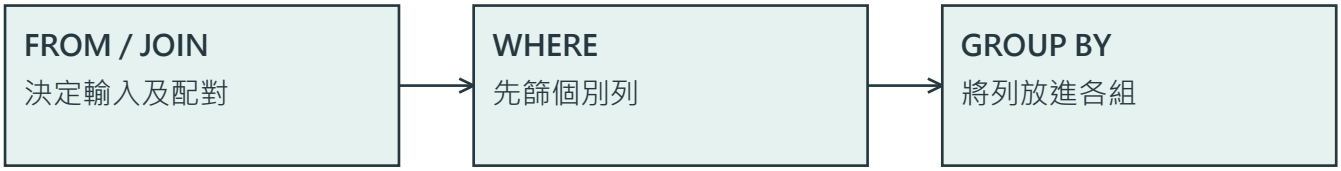
兩段查詢的實際完整結果。

答案：A 用 (80 + 0 + 90 + 95 + 60 + 75)/6，約 66.67。B 用 DB101 的 53.75、CS102 的 90、NW103 的 95；S7 的 60 只高於該課平均。

本頁重點：a.CourseCode 與 b.CourseCode 相等，才是拿同一門課來算平均。

查詢的概念順序：先篩列，再篩組

用「平均至少 70」回顧讀法：先理解有哪些輸入與配對，再看分組、條件和輸出。不是要求資料庫按照這張圖逐步執行。



前三步決定平均數要使用哪些成績。



後三步決定留下哪些組以及如何顯示。

問題：WHERE Grade >= 70 放在哪一步造成問題？

回答：平均之前。DB101 的 0、60 被排除，算的已不是本題要求的全課平均。

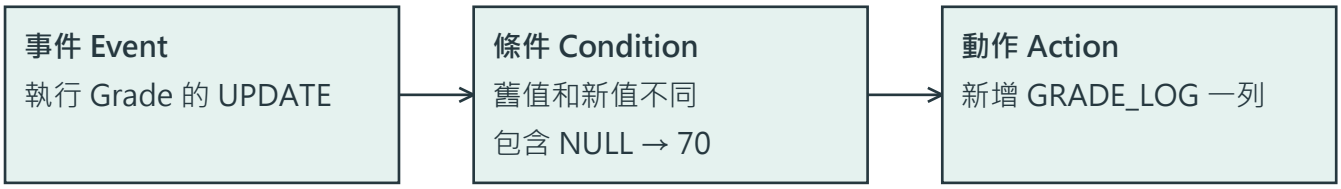
本頁重點：按概念順序解釋查詢；不要把此圖當成實際效能或執行計畫。

Trigger：成績異動時自動留下紀錄

教務處要知道哪一筆成績從多少改成多少。新增 GRADE_LOG 紀錄異動：學號、課程、舊成績、新成績；只有值真的改變才記錄。

StudentId	CourseCode	OldGrade	NewGrade
S1	CS102	NULL	70

這是要得到的紀錄範例，不是已修改原始資料。下一頁先建空紀錄表。



將事件、條件、動作寫成自動規則，稱為 Trigger。本例是 SQLite 實作額外補充。

本頁重點：Trigger 的任務是記成績異動；不是替每筆選課預先產生紀錄。

Trigger 準備：先建空的成績異動表

1. 重設學校起始資料。2. 在同一資料庫執行下方建表。此時八筆選課不變，GRADE_LOG 是空表。

```
CREATE TABLE GRADE_LOG (
  StudentId TEXT, CourseCode TEXT,
  OldGrade INTEGER, NewGrade INTEGER
);
```

GRADE_LOG：目前 0 列
StudentId、CourseCode、OldGrade、NewGrade

CREATE TABLE 只建紀錄表，不會自動有異動資料。

接下來 A、B 分別重做這兩步，再建立其中一個 Trigger，不要在同一資料庫重複建立同名規則。

本頁重點：紀錄表先準備好，Trigger 才能把新紀錄寫到它裡面。

成績異動 A：用 <> 判斷會漏掉 NULL

從 第46頁的空紀錄表開始。先建立本頁規則，還不修改成績。

OLD、NEW 分別是同一筆選課修改前、後的列。WHEN 是執行紀錄動作的條件；NULL <> 70 為 UNKNOWN，沒有通過。

```
CREATE TRIGGER RECORD_GRADE
AFTER UPDATE OF Grade ON ENROLLMENT
WHEN OLD.Grade <> NEW.Grade
BEGIN
  INSERT INTO GRADE_LOG VALUES
    (NEW.StudentId, NEW.CourseCode,
     OLD.Grade, NEW.Grade);
END;
```



規則建立完成，GRADE_LOG 仍為 0 列。

本頁重點：這頁只建立規則；第49頁再以同一筆 NULL → 70 比較紀錄結果。

成績異動 B：用 IS NOT 判斷包含 NULL

B 另從學校起始資料開始，只做第46頁建空紀錄表，再建立本頁規則。跳過第47頁，不要建立 A 的同名 Trigger。

只改 WHEN。SQLite 的 IS NOT 會將 NULL 與 70 判為不同；相同的 70 與 70 則判為相同，不新增紀錄。

```
CREATE TRIGGER RECORD_GRADE
AFTER UPDATE OF Grade ON ENROLLMENT
WHEN OLD.Grade IS NOT NEW.Grade
BEGIN
  INSERT INTO GRADE_LOG VALUES
    (NEW.StudentId, NEW.CourseCode,
     OLD.Grade, NEW.Grade);
END;
```



規則建立完成，GRADE_LOG 仍為 0 列。

本頁重點：這頁只建立規則；第49頁再以同一筆 NULL → 70 比較紀錄結果。

NULL 改成 70：同一段操作比較兩個 Trigger

目標是 S1 / CS102 從 NULL 改成 70 時必須記一筆。A：重設後做第46頁、第47頁，再執行本頁。B：另重設後做第46頁、第48頁，跳過第47頁，再執行本頁。

先猜：兩版的成績都會變 70，但紀錄列數一樣嗎？下方程式在各版各執行一次。

```
UPDATE ENROLLMENT SET Grade = 70
WHERE StudentId = 'S1' AND CourseCode = 'CS102';
SELECT Grade FROM ENROLLMENT
WHERE StudentId='S1' AND CourseCode='CS102';
SELECT * FROM GRADE_LOG ORDER BY StudentId, CourseCode;
```

Grade
70

第一個 SELECT：A、B 都回傳這一列，確認成績已改成 70。

A：不符合本題目標

0 列
沒有結果列

B：修正後

學號	課程	舊值	新值
S1	CS102	NULL	70

第二個 SELECT：GRADE_LOG 的完整結果。A 沒有列，B 有一列；不同的是紀錄，不是成績。

問題：為什麼 A 沒報錯，卻沒有紀錄？

回答：UPDATE 合法；WHEN 的 <> 遇 NULL 得 UNKNOWN，不是 TRUE，所以沒有執行 INSERT。

本頁重點：不只查成績有沒有改成功，還要查自動紀錄是否包含本題要求的異動。

成績異動後：八筆選課只改哪一列？

接續前頁 B；只有 S1 / CS102 從 NULL 變 70，其餘七筆不變。本頁列出全表，避免把紀錄表當成新的成績來源。

```
SELECT StudentId, CourseCode, Grade
FROM ENROLLMENT ORDER BY StudentId, CourseCode;
```

學號	課程	成績
S1	CS102	70
S1	DB101	80
S2	DB101	0
S6	CS102	90
S6	NW103	95
S7	DB101	60
S8	DB101	75
S9	CS102	NULL

操作後 ENROLLMENT 全八筆。S9 / CS102 仍是 NULL。

問題：S9 / CS102 也是 NULL，為什麼沒變？

回答：UPDATE 的 WHERE 只選中 S1 / CS102；Trigger 的 NEW 也只指這一筆。

接續 B 再設一次70；先預測紀錄會變成兩筆嗎？

```
UPDATE ENROLLMENT SET Grade = 70
WHERE StudentId = 'S1' AND CourseCode = 'CS102';
SELECT COUNT(*) FROM GRADE_LOG;
```

COUNT(*)
1

仍1筆：70沒有改變，WHEN 不成立，不新增紀錄。

本頁重點：GRADE_LOG 記錄異動；成績本身仍保存在 ENROLLMENT。

View：保存 DB101 的查詢

重設學校資料。教務處常查 DB101，將這段 SELECT 定義為 DB_GRADES。普通 View 保存查詢定義，不是凍結目前四筆結果。

```
CREATE VIEW DB_GRADES AS
SELECT StudentId, CourseCode, Grade
FROM ENROLLMENT
WHERE CourseCode = 'DB101';
```

```
SELECT * FROM DB_GRADES ORDER BY StudentId;
```

StudentId	CourseCode	Grade
S1	DB101	80
S2	DB101	0
S7	DB101	60
S8	DB101	75

依序執行建立、查詢兩段；完整結果四列。

本頁重點：View 有名稱，可以像表一樣查詢；結果仍來自原選課表。

View 與副本比較：先保存同時點的資料

1. 重設學校資料。2. 執行 第51頁的 CREATE VIEW。3. 執行下方程式保存四筆副本。現在 View 與副本相同，尚未改成績。

CREATE TABLE ... AS SELECT 把查詢結果存成普通表；本題取名 DB_SNAPSHOT。

```
CREATE TABLE DB_SNAPSHOT AS
SELECT * FROM DB_GRADES;
```

DB_GRADES：查詢結果

學號	課程	成績
S1	DB101	80
S2	DB101	0
S7	DB101	60
S8	DB101	75

DB_SNAPSHOT：副本

學號	課程	成績
S1	DB101	80
S2	DB101	0
S7	DB101	60
S8	DB101	75

此時兩邊都為四列，不能靠這一刻判斷哪個是 View。

先預測：下一頁只把原表 S1 / DB101 改成 90，View 與副本會各顯示 80 還是 90？

本頁重點：比較開始時兩者內容相同；後續只更新原表，不替副本另寫同步程式。

改成績：View 會變，副本不會自動變

接續 第52頁。先將 S1 / DB101 的原表成績 80 改成 90，再查兩份清單；沒有重設。

```
UPDATE ENROLLMENT SET Grade = 90
WHERE StudentId = 'S1' AND CourseCode = 'DB101';
```

```
SELECT * FROM DB_GRADES ORDER BY StudentId;
SELECT * FROM DB_SNAPSHOT ORDER BY StudentId;
```

DB_GRADES：現在是 90

學號	課程	成績
S1	DB101	90
S2	DB101	0
S7	DB101	60
S8	DB101	75

DB_SNAPSHOT：仍是 80

學號	課程	成績
S1	DB101	80
S2	DB101	0
S7	DB101	60
S8	DB101	75

兩段查詢的完整結果；只有 S1 的 90 / 80 不同。

本頁重點：普通 View 再查會讀原表現況；副本保存當時值，沒有自動同步。

修改成績：不要直接修改 SQLite 普通 View

把 S1 / DB101 改為 90。 A、B 各重設為開場資料。每邊重設後，先執行 第51頁的 CREATE VIEW，再執行自己的 A 或 B。

A：A 能成功改值嗎？B 再查 View 會看到 80 還是 90？ 先預測，再執行。

```
UPDATE DB_GRADES SET Grade = 90 WHERE StudentId = 'S1';
```

A 的實際錯誤：cannot modify DB_GRADES because it is a view

B：同一問題，改用以下寫法。

```
UPDATE ENROLLMENT SET Grade = 90
WHERE StudentId = 'S1' AND CourseCode = 'DB101';
SELECT * FROM DB_GRADES ORDER BY StudentId;
```

A：不符合本題目標

執行失敗
原始資料未改動

B：修正後

學號	課程	成績
S1	DB101	90
S2	DB101	0
S7	DB101	60
S8	DB101	75

A 的實際錯誤與 B 的完整結果。

答案：A 回報 cannot modify DB_GRADES because it is a view，沒有異動；B 修改原表，再查 View 看見 90。

本頁重點：本例未另設可修改 View 的 Trigger，SQLite 普通 View 為唯讀；不能推成所有 DBMS 的 View 都不可修改。

DROP VIEW：移除查詢定義，不刪選課

重設學校資料，依 第51頁建立 DB_GRADES。目標是移除查詢名稱，不刪 DB101 選課。先預測哪一段 SELECT 會失敗。

```
DROP VIEW DB_GRADES;
```

```
SELECT * FROM DB_GRADES;
```

實際錯誤：no such table: DB_GRADES。名稱已移除；報錯後另執行下方查原表，不重做 DROP。

```
SELECT StudentId, CourseCode, Grade FROM ENROLLMENT
WHERE CourseCode = 'DB101' ORDER BY StudentId;
```

學號	課程	成績
S1	DB101	80
S2	DB101	0
S7	DB101	60
S8	DB101	75

DB101 的完整四筆仍在；DB_GRADES 名稱已移除。

本頁重點：DROP VIEW 不等於 DELETE 選課資料。

清空選課或移除選課表：DELETE 與 DROP

A、B 各重設學校資料。本題目標是清空選課，但保留表供下次 INSERT；沒有 View 或 Trigger 依賴此表。

A：先移除表，再查列數。

```
DROP TABLE ENROLLMENT;
```

```
SELECT COUNT(*) FROM ENROLLMENT;
```

B：另重設，先清空列，再查列數。

```
DELETE FROM ENROLLMENT;
SELECT COUNT(*) FROM ENROLLMENT;
```

A：查詢失敗
no such table: ENROLLMENT
表與主鍵限制也移除

B：COUNT(*) = 0
完整輸出一列：0
空表及原限制仍保留

兩邊都沒有選課資料，但只有 B 還有 ENROLLMENT 表可繼續使用。

問題：下次要再 INSERT，哪一版不用重建表？

回答：B。A 已移除表，直接 INSERT 會找不到表。

本頁重點：DELETE 針對資料列；DROP TABLE 針對整張表的定義與資料。

ADD COLUMN：新增 Email，舊八人仍沒有信箱值

重設學校資料。新需求要保存學生 Email；只新增欄位，沒有任何來源能替八人猜出信箱。

```
ALTER TABLE STUDENT ADD COLUMN Email TEXT;
SELECT StudentId, Email FROM STUDENT ORDER BY StudentId;
```

StudentId	Email
S1	NULL
S2	NULL
S3	NULL
S5	NULL
S6	NULL
S7	NULL
S8	NULL
S9	NULL

完整八人新欄位均為 NULL；學號、姓名、系所未變。

本頁重點：新增欄位不等於已取得資料；沒有預設值時，既有列的新欄為 NULL。

ADD COLUMN：用 DEFAULT 指定既有列的起始值

新增必填 ContactState，舊學生先標為 pending。 A、B 各重設為開場資料。 ContactState = 聯絡資料確認狀態；pending = 尚待確認。DEFAULT 指定未填值時採用的預設值。

A：A 能加欄嗎？B 完成後，八人的新欄位會是什麼？ 先預測，再執行。

```
ALTER TABLE STUDENT ADD COLUMN ContactState TEXT NOT NULL;
```

A 的實際錯誤：Cannot add a NOT NULL column with default value NULL

B：同一問題，改用以下寫法。

```
ALTER TABLE STUDENT ADD COLUMN ContactState
  TEXT NOT NULL DEFAULT 'pending';
SELECT StudentId, ContactState FROM STUDENT ORDER BY StudentId;
```

A：不符合本題目標

執行失敗
原始資料未改動

B：修正後

學號	狀態
S1	pending
S2	pending
S3	pending
S5	pending
S6	pending
S7	pending
S8	pending
S9	pending

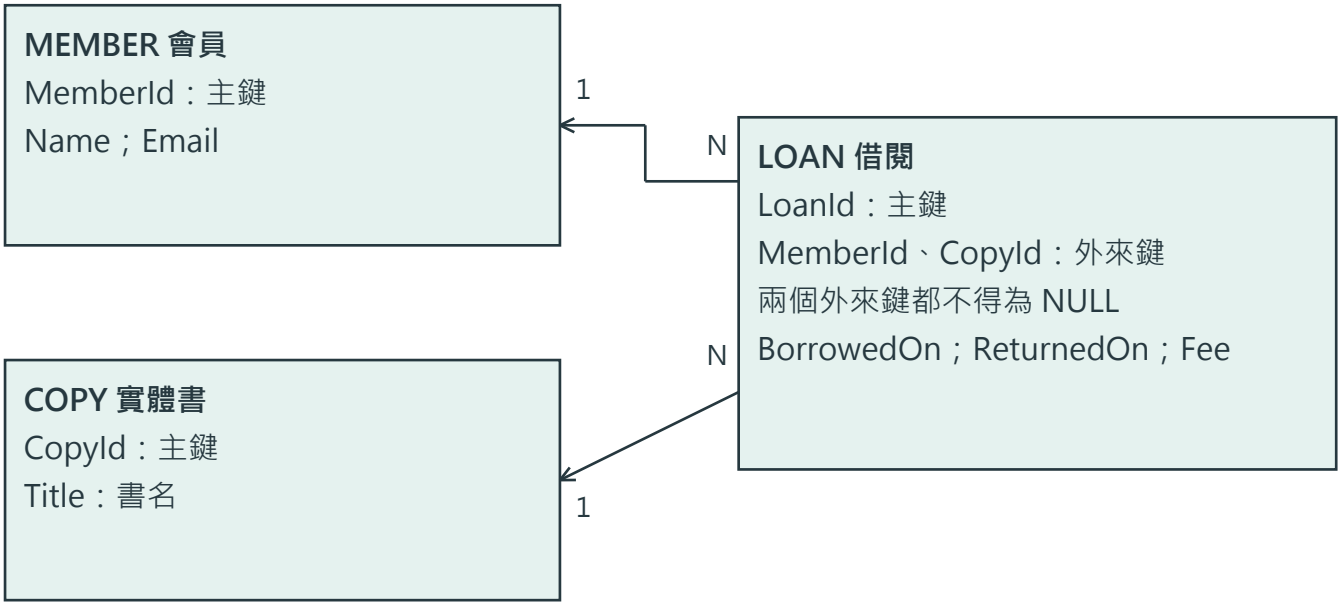
A 的實際錯誤與 B 的完整結果。

答案：A 執行失敗，表未改；B 八人都有 pending，表示尚待確認，不是已確認。

本頁重點：SQLite 實作補充：非空表新增 NOT NULL 欄位，必須給可使用的非 NULL 預設值。

圖書館資料：會員、實體書與借閱事件

館員要查誰借過書、哪些尚未歸還，以及每次借閱的費用。沿用 Ch6 四位會員、四本實體書、六次借閱；每次都由 LoanId 識別。



同一會員可借同一本書多次；每筆借閱連一位會員與一本實體書。

本章新增 Fee（元）：館員確認的費用；NULL 是尚待核對，0 是已確認免收。這些為人工費用值，不按借閱天數計算。

本頁重點：先識別哪次借閱，再看歸還與費用欄位；兩個欄位的缺值意義不同。

完整起始資料：四位會員與四本實體書

M 是會員編號，B 是實體書編號。M1、M3 都叫小安；B1、B4 同名但不是同一本實體書。

MemberId	Name	Email
M1	小安	NULL
M2	小文	wen@lib.test
M3	小安	ning@lib.test
M4	小青	NULL

M4 小青尚未借閱。Email 的 NULL 是尚無信箱資料。

CopyId	Title
B1	SQL入門
B2	SQL_練習
B3	DB設計
B4	SQL入門

四本書全部列出；兩本 SQL入門也要分別用 B1、B4。

本頁重點：會員與書都用編號識別，不用重複的姓名或書名合併。

完整起始資料：六次借閱的日期

L 是借閱事件編號。BorrowedOn = 借出日，ReturnedOn = 歸還日；日期採 YYYY-MM-DD。ReturnedOn 為 NULL 表示此例尚未歸還。

LoanId	MemberId	CopyId
L1	M1	B1
L2	M1	B1
L3	M2	B2
L4	M3	B3
L5	M2	B4
L6	M3	B1

六筆借閱的身分欄位；下圖為同六筆的日期欄位，不是另一份資料。

LoanId	BorrowedOn	ReturnedOn
L1	2026-09-01	2026-09-05
L2	2026-09-10	2026-09-15
L3	2026-09-11	NULL
L4	2026-09-04	2026-09-08
L5	2026-09-06	2026-09-09
L6	2026-09-16	NULL

L1、L2 是 M1 先後借 B1；L3、L6 尚未歸還。

本頁重點：不要只用會員加書號識別一次借閱；同一組可以多次發生。

完整起始資料：借閱費用的 0 與 NULL

同六次借閱。館員已確認 L1 收 20 元、L2 免收、L4 收 10 元；L3、L5、L6 尚待核對。L5 已歸還，仍可能費用未知。

LoanId	ReturnedOn	Fee（元）
L1	2026-09-05	20
L2	2026-09-15	0
L3	NULL	NULL
L4	2026-09-08	10
L5	2026-09-09	NULL
L6	NULL	NULL

日期與金額來源是同六筆 LOAN；0 已確認，NULL 未確認。

另開空 SQLite 資料庫，依 第83頁到第86頁四步建置。後續題目從這六筆開始；只查詢時可接續執行，修改型比較則各自重設。

本頁重點：費用未知不等於未還書，也不等於免收費。

缺值再應用：未還書與費用未知

比較「尚未還書」和「費用尚待核對」的借閱清單。 從開場資料開始；A、B 都只查詢，可接續執行。

A：L5 已還書但費用未知；會在哪份清單中？ 先預測，再執行。

```
SELECT LoanId FROM LOAN WHERE ReturnedOn IS NULL
ORDER BY LoanId;
```

B：改查費用欄位，回答另一個問題。

```
SELECT LoanId FROM LOAN WHERE Fee IS NULL
ORDER BY LoanId;
```

A：尚未還書

借閱
L3
L6

B：費用尚待核對

借閱
L3
L5
L6

兩段查詢的實際完整結果。

答案：兩版都是合法且符合各自問題的查詢，不是 A 錯 B 對。L5 已還但費用未知，只有第二份清單有 L5。

本頁重點：同樣是 IS NULL，問的欄位不同，回答的問題也不同。

費用條件：排除免收，但保留尚待核對

列出非 0 元或費用未知的借閱。 從開場資料開始；A、B 都只查詢，可接續執行。

A：兩版會留下費用未知的 L3、L5、L6 嗎？ 先預測，再執行。

```
SELECT LoanId, Fee FROM LOAN
WHERE NOT (Fee = 0) ORDER BY LoanId;
```

B：同一問題，改用以下寫法。

```
SELECT LoanId, Fee FROM LOAN
WHERE NOT (Fee = 0) OR Fee IS NULL ORDER BY LoanId;
```

A：不符合本題目標

借閱	費用
L1	20
L4	10

B：修正後

借閱	費用
L1	20
L3	NULL
L4	10
L5	NULL
L6	NULL

兩段查詢的實際完整結果。

答案：A 只有兩筆已知非零費用，漏掉三筆尚待核對。B 依本題目標保留五筆，只排除 L2 的 0。

本頁重點：NULL 不會因 NOT 而變成成立；要保留尚待核對，另寫 IS NULL。

借閱會員名單：不要把借閱次數當人數

每位借過書的會員只列一次。 從開場資料開始；A、B 都只查詢，可接續執行。m 是 MEMBER 的別名；ln（小寫 L 加 n）是 LOAN 的別名，不是關鍵字 IN。

M1、M2、M3 各借過兩次，M4 沒有借閱；A、B 各列出幾列？先預測，再執行 A。

```
SELECT m.MemberId, m.Name FROM MEMBER AS m
JOIN LOAN AS ln ON m.MemberId = ln.MemberId
ORDER BY m.MemberId, ln.LoanId;
```

B：同一問題，改用以下寫法。

```
SELECT MemberId, Name FROM MEMBER
WHERE MemberId IN (SELECT MemberId FROM LOAN)
ORDER BY MemberId;
```

A：不符合本題目標

會員	姓名
M1	小安
M1	小安
M2	小文
M2	小文
M3	小安
M3	小安

B：修正後

會員	姓名
M1	小安
M2	小文
M3	小安

兩段查詢的實際完整結果。

答案：A 得六次借閱配對；B 得三位會員。M1、M3 同名仍是兩人，不要只列 DISTINCT Name。

本頁重點：先決定要列會員還是借閱事件，再選擇名單比對或接合。

未借閱會員：NOT EXISTS 與 NOT IN

找出沒有任何借閱的會員。 從開場資料開始；A、B 都只查詢，可接續執行。

A：哪位會員沒有任何借閱？兩版會相同嗎？ 先預測，再執行。

```
SELECT MemberId FROM MEMBER
WHERE MemberId NOT IN (SELECT MemberId FROM LOAN)
ORDER BY MemberId;
```

B：同一問題，改用以下寫法。

```
SELECT m.MemberId FROM MEMBER AS m
WHERE NOT EXISTS (SELECT 1 FROM LOAN AS ln
  WHERE ln.MemberId = m.MemberId) ORDER BY m.MemberId;
```

A：NOT IN，正確

會員
M4

B：NOT EXISTS，正確

會員
M4

兩段查詢的實際完整結果。

答案：兩版都得 M4。LOAN.MemberId 已設 NOT NULL；M2、M3 雖有未還書，也不是從未借閱。

本頁重點：沒有借閱事件，與有事件但未歸還，不是同一條件。

全部會員的待核費用：條件放 ON

列出全部會員，只配對費用未知的借閱。 從開場資料開始；A、B 都只查詢，可接續執行。M1 的兩筆費用是 20、0；M4 沒有借閱。用這兩人檢查是否真的保留全部會員。

A：M1 費用皆已知，M4 未借閱；兩版會保留誰？ 先預測，再執行。

```
SELECT m.MemberId, ln.LoanId FROM MEMBER AS m
LEFT JOIN LOAN AS ln ON m.MemberId = ln.MemberId
WHERE ln.Fee IS NULL ORDER BY m.MemberId, ln.LoanId;
```

B：同一問題，改用以下寫法。

```
SELECT m.MemberId, ln.LoanId FROM MEMBER AS m
LEFT JOIN LOAN AS ln ON m.MemberId = ln.MemberId
  AND ln.Fee IS NULL ORDER BY m.MemberId, ln.LoanId;
```

A：不符合本題目標

會員	借閱
M2	L3
M2	L5
M3	L6
M4	NULL

B：修正後

會員	借閱
M1	NULL
M2	L3
M2	L5
M3	L6
M4	NULL

兩段查詢的實際完整結果。

答案：A 漏了 M1：他的兩筆 Fee 都已知。M4 沒有配對，LoanId、Fee 都補 NULL，因此 Fee IS NULL 成立；不代表他有待核借閱。B 五列包含四位會員，達成本題目標。

本頁重點：ON 先限定配對；沒有待核借閱的會員仍保留一列，LoanId 補 NULL。

會員借閱數：COUNT(*) 與 COUNT(LoanId)

計算每位會員的實際借閱次數，從未借閱為 0。 從開場資料開始；A、B 都只查詢，可接續執行。

A：M4 未借閱；兩版會各算成 0 還是 1？ 先預測，再執行。

```
SELECT m.MemberId, COUNT(*) FROM MEMBER AS m
LEFT JOIN LOAN AS ln ON m.MemberId = ln.MemberId
GROUP BY m.MemberId ORDER BY m.MemberId;
```

B：同一問題，改用以下寫法。

```
SELECT m.MemberId, COUNT(ln.LoanId) FROM MEMBER AS m
LEFT JOIN LOAN AS ln ON m.MemberId = ln.MemberId
GROUP BY m.MemberId ORDER BY m.MemberId;
```

A：不符合本題目標

會員	借閱數
M1	2
M2	2
M3	2
M4	1

B：修正後

會員	借閱數
M1	2
M2	2
M3	2
M4	0

兩段查詢的實際完整結果。

答案：M4 沒有 LOAN 列；A 計到了補出列，B 得 0。計 Fee 則是在數已核對費用，不是借閱數。

本頁重點：數右表的非 NULL 識別欄，才能分辨實際配對與補出列。

費用平均：30 元除以六次，還是三筆已知？

計算已確認費用的平均，免收 0 元也要計入。 從開場資料開始；A、B 都只查詢，可接續執行。

A：已知費用合計 30；兩版各除以幾筆？ 先預測，再執行。

```
SELECT SUM(Fee) * 1.0 / COUNT(*) AS Mean FROM LOAN;
```

B：同一問題，改用以下寫法。

```
SELECT AVG(Fee) AS Mean FROM LOAN;
```

A：不符合本題目標

平均（元）
5.0

B：修正後

平均（元）
10.0

兩段查詢的實際完整結果。

答案：已知費用是 20、0、10，加總 30。A 除全部六次得 5；B 除三筆已知值得 10。

本頁重點：免收 0 元是已知值；尚待核對的 NULL 不作為分母。

會員已確認費用至少 15：保留借閱再統計

找已確認費用合計至少 15 元的會員，列全部借閱數與已知平均。 從開場資料開始；A、B 都只查詢，可接續執行。

A：M1 有 20 與 0 元兩筆；兩版的次數、平均各多少？ 先預測，再執行。

```
SELECT MemberId, COUNT(*), AVG(Fee) FROM LOAN
WHERE Fee >= 15 GROUP BY MemberId ORDER BY MemberId;
```

B：同一問題，改用以下寫法。

```
SELECT MemberId, COUNT(*), AVG(Fee) FROM LOAN
GROUP BY MemberId HAVING SUM(Fee) >= 15
ORDER BY MemberId;
```

A：不符合本題目標

會員	借閱數	平均
M1	1	20.0

B：修正後

會員	借閱數	平均
M1	2	10.0

兩段查詢的實際完整結果。

答案：A 先去掉 M1 的 L2 免收紀錄，只算一筆 20。B 保留 L1、L2，總費 20、借閱兩次、平均 10。

本頁重點：本題只加總已確認金額；M2、M3 還有待核費用，不代表真實總費已知。

高於整體平均：費用子查詢再應用

接續前頁純查詢，仍是六筆原借閱，不需重設。先算平均，再預測哪些 LoanId 會留下。

```
SELECT AVG(Fee) FROM LOAN;
```

AVG(Fee)
10.0

內層完整輸出1列：(20 + 0 + 10) / 3 = 10元；NULL 不參與平均。

```
SELECT LoanId, Fee FROM LOAN
WHERE Fee > (SELECT AVG(Fee) FROM LOAN) ORDER BY LoanId;
```

LoanId	Fee (元)	> 10 ?
L1	20	是
L2	0	否
L3	NULL	UNKNOWN
L4	10	否：等於
L5	NULL	UNKNOWN
L6	NULL	UNKNOWN

逐列對照：只有 L1 的判斷為 TRUE。

LoanId	Fee
L1	20

外層完整輸出1列；內層算平均的 SELECT 不會更改資料。

本頁重點：內層回傳平均 10，外層比較每次借閱費用；10 不算高於 10。

View 再應用：保存費用待核清單

重設六筆借閱。建立 PENDING_FEE 保存待核費用查詢；其中也包含已還書的 L5，不等於未還書清單。

```
CREATE VIEW PENDING_FEE AS
SELECT LoanId, MemberId, Fee
FROM LOAN WHERE Fee IS NULL;
```

```
SELECT * FROM PENDING_FEE ORDER BY LoanId;
```

LoanId	MemberId	Fee
L3	M2	NULL
L5	M2	NULL
L6	M3	NULL

依序建 View、查 View，完整結果三列。

本頁重點：View 的條件仍是 Fee IS NULL，名稱不能替代明確的篩選規則。

費用核對後：View 更新，當時副本保留 L5

1. 重設圖書館並建立 第72頁的 View。2. 保存目前三筆副本。3. 確認 L5 費用 30 元。4. 查兩份清單；本題修改後不要重設。

```
CREATE TABLE FEE_SNAPSHOT AS SELECT * FROM PENDING_FEE;
```

```
UPDATE LOAN SET Fee = 30 WHERE LoanId = 'L5';
```

```
SELECT * FROM PENDING_FEE ORDER BY LoanId;
SELECT * FROM FEE_SNAPSHOT ORDER BY LoanId;
```

PENDING_FEE：兩列

借閱	會員	費用
L3	M2	NULL
L6	M3	NULL

FEE_SNAPSHOT：三列

借閱	會員	費用
L3	M2	NULL
L5	M2	NULL
L6	M3	NULL

兩份清單的完整結果；L5 已核對，所以不在當前待核 View。

本頁重點：副本仍保留當時 L5 的未知值；原表現在已是 30，不能把副本當作現況。

費用核對後：六筆借閱只改 L5

接續前頁，不重設。L5 的 Fee 從 NULL 改成 30，借閱者、實體書、借出和歸還日都沒改。

```
SELECT LoanId, MemberId, CopyId, Fee FROM LOAN
ORDER BY LoanId;
```

借閱	會員	實體書	Fee（元）
L1	M1	B1	20
L2	M1	B1	0
L3	M2	B2	NULL
L4	M3	B3	10
L5	M2	B4	30
L6	M3	B1	NULL

全六筆身分與費用；待核只剩 L3、L6。歸還日不是本段 SELECT 輸出。

本頁重點：先查原表異動的那一列，再解釋為什麼它會從 View 消失。

新增會員聯絡備註：ADD COLUMN 後再填值

另開新資料庫，依 第83頁到第86頁完整重設；尚無 ContactNote。這是會員聯絡備註欄，新增後只替 M4 填入待聯絡。

```
ALTER TABLE MEMBER ADD COLUMN ContactNote TEXT;
```

```
UPDATE MEMBER SET ContactNote = '待聯絡'
WHERE MemberId = 'M4';
SELECT MemberId, ContactNote FROM MEMBER ORDER BY MemberId;
```

會員	ContactNote
M1	NULL
M2	NULL
M3	NULL
M4	待聯絡

兩段按順序執行；完整四人結果，只有 M4 有填入值。

本頁重點：ALTER 新增欄位，UPDATE 才填入已取得的資料；不會替其他人猜值。

查詢與異動：依目標選擇 SQL

沿用兩個故事回顧：先問要列人、列事件、數列，還是更新資料，再選寫法。不要只背關鍵字的位置。

要回答的問題	檢查的資料	對應寫法
S9 成績登錄了嗎？	S9 / CS102：NULL	IS NULL
哪些學生沒選課？	S3、S5 沒有配對	NOT EXISTS
全部學生都要出現？	補出的 S3、S5	LEFT JOIN
CS102 的已知平均？	NULL、90、NULL	AVG：90 / 1
整門課平均至少 70？	先分組，不先篩成績	GROUP BY / HAVING
NULL 改 70 要自動記？	舊值、新值、紀錄表	Trigger
保存待核費用的查詢？	L3、L5、L6	View
新欄位怎麼有資料？	先加欄，再填已知值	ALTER / UPDATE

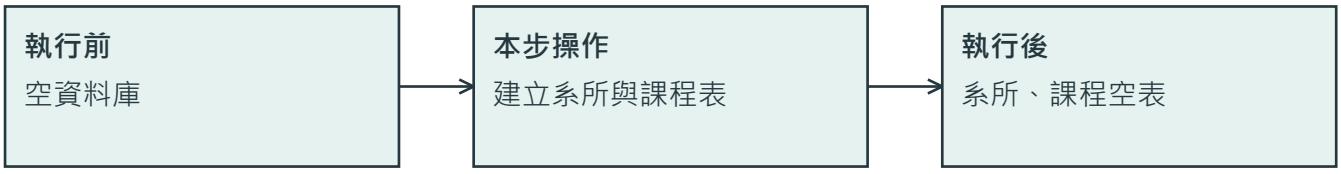
每列都可回到前面完整資料與實際結果驗證。

本頁重點：先說清楚目標和資料，再選 SQL；合法程式不一定回答了原本的問題。

學校建置 1：建立系所與課程表

先開空 SQLite 資料庫，啟用外來鍵檢查，再建兩張主鍵表。

```
PRAGMA foreign_keys = ON;
CREATE TABLE DEPT (
  DeptId TEXT NOT NULL PRIMARY KEY,
  DeptName TEXT NOT NULL UNIQUE
);
CREATE TABLE COURSE (
  CourseCode TEXT NOT NULL PRIMARY KEY
);
PRAGMA foreign_keys;
```



本步完成後，保留同一連線再接下一步。

最後一行 PRAGMA foreign_keys; 的實際輸出是 1。實作額外補充：PRAGMA foreign_keys = ON 是此連線的開關；哪些欄位是外來鍵，仍要在 REFERENCES 逐一指定。

本頁重點：六步是準備順序；做題目時使用完成後的固定資料，不混用未建好或改過的表。

學校建置 2：建立學生表

接續前頁。省略 DeptId 會填入 D1；要保持尚未分系，INSERT 時須明確寫 NULL。

```
CREATE TABLE STUDENT (
  StudentId TEXT NOT NULL PRIMARY KEY,
  Name TEXT NOT NULL,
  DeptId TEXT DEFAULT 'D1',
  FOREIGN KEY (DeptId) REFERENCES DEPT(DeptId)
    ON DELETE RESTRICT ON UPDATE CASCADE
);
```



本步完成後，保留同一連線再接下一步。

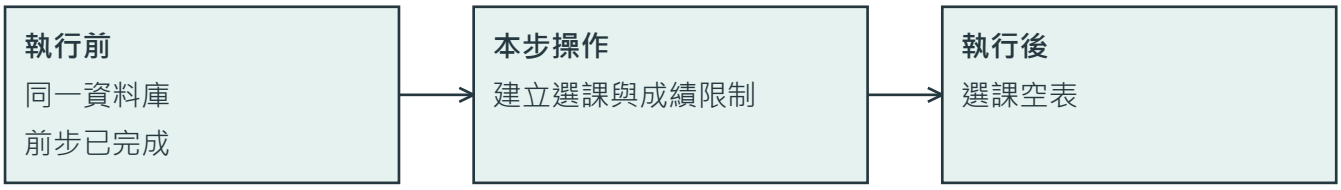
完成本步後，保留同一連線與已有資料；起始資料內容均已在開場全表列出。

本頁重點：六步是準備順序；做題目時使用完成後的固定資料，不混用未建好或改過的表。

學校建置 3：建立選課與成績限制

接續前兩頁；每位學生每門課只有一列，Grade 限 0–100 整數或 NULL。

```
CREATE TABLE ENROLLMENT (
  StudentId TEXT NOT NULL REFERENCES STUDENT(StudentId),
  CourseCode TEXT NOT NULL REFERENCES COURSE(CourseCode),
  Grade INTEGER,
  PRIMARY KEY (StudentId, CourseCode),
  CONSTRAINT grade_domain CHECK (Grade IS NULL OR
    (typeof(Grade) = 'integer' AND Grade BETWEEN 0 AND 100))
);
```



本步完成後，保留同一連線再接下一步。

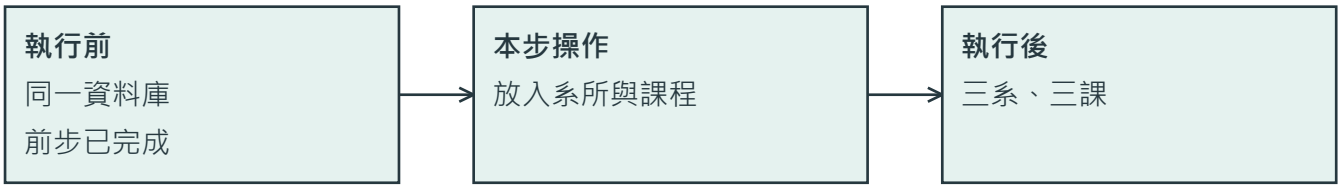
實作額外補充：typeof 檢查 SQLite 實際儲存型別；此成績規則不是只宣告 INTEGER 就會自動保證。

本頁重點：六步是準備順序；做題目時使用完成後的固定資料，不混用未建好或改過的表。

學校建置 4：放入系所與課程

表已建好但都空。先放三系三課，才有主鍵可供學生與選課參照。

```
INSERT INTO DEPT VALUES ('D1','資訊管理'),('D2','會計'),('D3','設計');
INSERT INTO COURSE VALUES ('DB101'),('CS102'),('NW103');
```



本步完成後，保留同一連線再接下一步。

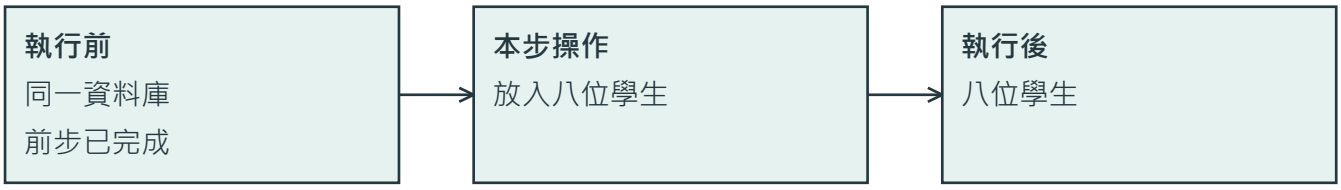
完成本步後，保留同一連線與已有資料；起始資料內容均已在開場全表列出。

本頁重點：六步是準備順序；做題目時使用完成後的固定資料，不混用未建好或改過的表。

學校建置 5：放入八位學生

接續前四步；姓名重複允許，兩位尚未分系的人明確填 NULL。

```
INSERT INTO STUDENT(StudentId,Name,DeptId) VALUES
  ('S1','Alex','D1'),('S2','Alex','D1'),
  ('S3','Casey',NULL),('S5','Morgan','D1'),
  ('S6','Riley','D2'),('S7','Taylor','D2'),
  ('S8','Jordan','D2'),('S9','Robin',NULL);
```



本步完成後，保留同一連線再接下一步。

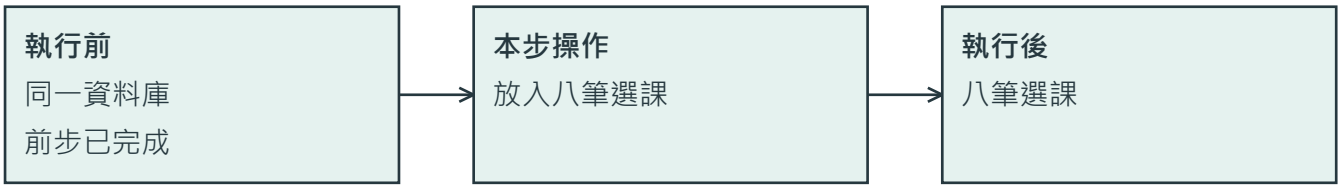
完成本步後，保留同一連線與已有資料；起始資料內容均已在開場全表列出。

本頁重點：六步是準備順序；做題目時使用完成後的固定資料，不混用未建好或改過的表。

學校建置 6：放入八筆選課

接續前五步；每筆學號與課程已存在，0 與 NULL 的意義不同。

```
INSERT INTO ENROLLMENT(StudentId,CourseCode,Grade) VALUES
  ('S1','DB101',80),('S1','CS102',NULL),
  ('S2','DB101',0),('S6','CS102',90),
  ('S7','DB101',60),('S8','DB101',75),
  ('S9','CS102',NULL),('S6','NW103',95);
```



六步已完成，保留同一連線，回到學校查詢題。

完成本步後，保留同一連線與已有資料；起始資料內容均已在開場全表列出。

本頁重點：六步是準備順序；做題目時使用完成後的固定資料，不混用未建好或改過的表。

圖書館建置 1：建立會員與實體書

另開空資料庫，不與學校表混在一起；本步只建兩張主鍵表。

```
PRAGMA foreign_keys = ON;
CREATE TABLE MEMBER (
  MemberId TEXT NOT NULL PRIMARY KEY,
  Name TEXT NOT NULL,
  Email TEXT UNIQUE CHECK (Email IS NULL OR Email <> '')
);
CREATE TABLE COPY (
  CopyId TEXT NOT NULL PRIMARY KEY,
  Title TEXT NOT NULL
);
```



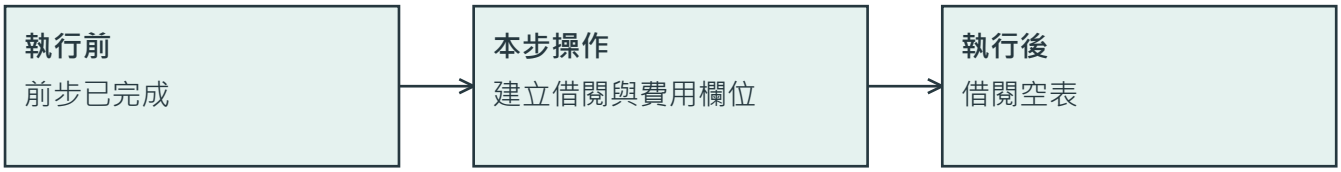
這四步在同一資料庫接續操作；完成後才開始查詢或修改資料。

本頁重點：準備步驟只放本步需要的程式；背景、代碼與完整起始表見圖書館開場。

圖書館建置 2：建立借閱與費用欄位

接續前頁同一資料庫；Fee 是人工範例新增的欄，不是借閱天數自動計算。

```
CREATE TABLE LOAN (
  LoanId TEXT NOT NULL PRIMARY KEY,
  MemberId TEXT NOT NULL REFERENCES MEMBER(MemberId)
    ON DELETE RESTRICT,
  CopyId TEXT NOT NULL REFERENCES COPY(CopyId),
  BorrowedOn TEXT NOT NULL, ReturnedOn TEXT,
  Fee INTEGER CHECK (Fee IS NULL OR
    (typeof(Fee) = 'integer' AND Fee >= 0)),
  CHECK (ReturnedOn IS NULL OR ReturnedOn >= BorrowedOn)
);
```



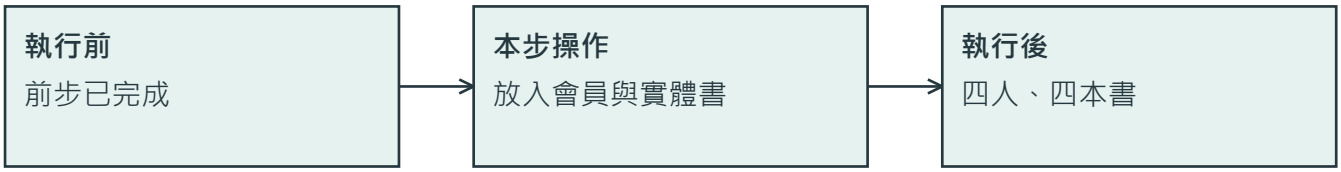
這四步在同一資料庫接續操作；完成後才開始查詢或修改資料。

本頁重點：準備步驟只放本步需要的程式；背景、代碼與完整起始表見圖書館開場。

圖書館建置 3：放入會員與實體書

表已建立，先放四位會員與四本實體書，才有主鍵可被借閱參照。

```
INSERT INTO MEMBER VALUES
  ('M1','小安',NULL),('M2','小文','wen@lib.test'),
  ('M3','小安','ning@lib.test'),('M4','小青',NULL);
INSERT INTO COPY VALUES
  ('B1','SQL入門'),('B2','SQL_練習'),
  ('B3','DB設計'),('B4','SQL入門');
```



這四步在同一資料庫接續操作；完成後才開始查詢或修改資料。

本頁重點：準備步驟只放本步需要的程式；背景、代碼與完整起始表見圖書館開場。

圖書館建置 4：放入六次借閱

接續前三步；日期與費用均依開場全表，不自行推算或改值。

```
INSERT INTO LOAN VALUES
  ('L1','M1','B1','2026-09-01','2026-09-05',20),
  ('L2','M1','B1','2026-09-10','2026-09-15',0),
  ('L3','M2','B2','2026-09-11',NULL,NULL),
  ('L4','M3','B3','2026-09-04','2026-09-08',10),
  ('L5','M2','B4','2026-09-06','2026-09-09',NULL),
  ('L6','M3','B1','2026-09-16',NULL,NULL);
```



四步已完成，回到圖書館查詢題；遇到修改型比較才各自重設。

本頁重點：準備步驟只放本步需要的程式；背景、代碼與完整起始表見圖書館開場。